Informe de la Etapa 2: Integridad y Lógica · Bodegas del Atuel

Trabajo Práctico Integrador · Bases de Datos · Ingeniería en Sistemas · UTN Facultad Regional San Rafael · Ciclo 2026.

---

1. Introducción y Alcance de la Etapa 2

La Etapa 2 representa el **núcleo funcional y de integridad del sistema (70% de la nota de la capa de datos)**. En esta etapa se implementaron todos los mecanismos de control transaccional, reglas de negocio explícitas (**RN-01 a RN-15**) e implícitas (**RI-01 a RI-07**), vistas analíticas y operativas, modelo de roles de seguridad con privilegios mínimos, y el banco de pruebas automáticas en SQL.

Los entregables correspondientes a esta etapa son:

1. `db/migrations/002_constraints.sql`: Restricciones de integridad adicionales (`UNIQUE`, `CHECK`, `DEFAULT`).
2. `db/migrations/003_funciones.sql`: Funciones de cálculo de stock, totales, antigüedad y descuentos.
3. `db/migrations/004_procedimientos.sql`: Procedimientos almacenados transaccionales para procesamiento de ingresos y transferencias.
4. `db/migrations/005_triggers.sql`: Triggers de protección de reglas de negocio (RN-05, RN-08, RN-09, RN-10, RN-11, RN-12, RN-13 y RI-01 a RI-06).
5. `db/migrations/006_vistas.sql`: Vistas de stock total, ventas mensuales (materializada), documentos pendientes y unificación de clientes.
6. `db/migrations/007_roles.sql`: Definición de roles (`app_rw`, `reportes_ro`) y asignación de privilegios mínimos.
7. `db/tests/db_tests.sql`: Script completo de pruebas de aceptación (casos válidos y casos inválidos esperados).

---

2. Cobertura de Reglas de Negocio (RN y RI)

Todas las reglas estipuladas en el enunciado y en el `SPEC.md` se encuentran protegidas en la base de datos:

Regla	Descripción	Mecanismo en la base
RN-01	Precio del vino único en la red y > 0	`CHECK (precio > 0)` + `UNIQUE (nombre, anada, volumen_ml)`
RN-02	Sala en una sola localidad; localidad con único encargado	FK + `UNIQUE (cuit_encargado)`

Regla	Descripción	Mecanismo en la base
RN-03	Empleado en una sola sala	`FK NOT NULL` en `empleado.id_sala`
RN-04	Parcela con una cepa; cosecha máx. una vez por campaña	Índice único de expresión en `lote_uva` (`id_finca, nro_parcela, EXTRACT(YEAR FROM fecha_cosecha)`)
RN-05	Porcentajes de corte suman 100	Constraint trigger diferido sobre `composicion`
RN-06	Descuento por tipo (socio 10%, distribuidor 20%, exportador 30%)	Trigger en `cliente` + `CHECK (descuento BETWEEN 0 AND 35)`
RN-07	Exportador con varios permisos; vencimiento > emisión	`CHECK (fecha_vencimiento > fecha_emision)` en `permiso_exportacion`
RN-08	Total del comprobante = suma de renglones con descuento	Funciones y triggers `AFTER INSERT/UPDATE` en `renglon_comprobante`
RN-09	No vender más que el stock de la sala; stock nunca negativo	Trigger `BEFORE INSERT` con `FOR UPDATE` + `CHECK (cantidad >= 0)`
RN-10	Solo ingreso terminado impacta el depósito; transiciones sin retroceso	ENUM + trigger de control de estados en `ingreso`
RN-11	Ídem transferencias; fecha de confección = actual e inmutable	Trigger de estados y fecha inmutable en `transferencia`
RN-12	Documento no en confección no admite tocar renglones	Trigger `BEFORE INSERT/UPDATE/DELETE` en renglones logísticos
RN-13	Cambio de precio auditado (fecha, usuario, precio anterior/nuevo)	Trigger `AFTER UPDATE` sobre `vino` + tabla `auditoria_precio`
RN-14	No borrar transportista/bodega/vino con movimientos	`FK ON DELETE RESTRICT`
RN-15	Cantidades, superficies, kg y precios > 0; graduación 5–20	`CHECK` constraints en tablas respectivas
RI-01	El encargado trabaja en una sala de su misma localidad	Constraint trigger diferido
RI-02	Todo vino tiene al menos un maridaje	Constraint trigger diferido

Regla	Descripción	Mecanismo en la base
RI-03	El precio unitario es una copia histórica de `vino.precio`	Trigger `BEFORE INSERT` en `renglon_comprobante`
RI-04	Los vinos de un ingreso son de la bodega de origen	Trigger en `renglon_ingreso`
RI-05	El transportista cubre la localidad del depósito	Trigger en `ingreso`
RI-06	Exportador con más de 3 permisos recibe 35 %	Trigger en `permiso_exportacion` recalculando descuento
RI-07	El subtipo coincide con el tipo del cliente	FK compuesta `(id_cliente, tipo)`

---

3. Demo de Concurrencia (Gestión de la Última Botella)

3.1 Planteo del Problema y Concurrencia

En sistemas transaccionales con alta concurrencia de ventas (como una red de salas de venta distribuidas), dos cajeros podrían intentar vender simultáneamente la última botella disponible de un vino (`stock = 1`).

Sin el control de concurrencia adecuado (bajo el nivel de aislamiento por defecto de PostgreSQL `Read Committed`), ambas sesiones leerían `stock = 1` en su consulta inicial, validarían que la cantidad solicitada (`1`) es menor o igual al stock disponible, y procederían a descontar el stock, provocando una **sobreventa** (stock negativo o inconsistencia de inventario).

3.2 Solución Implementada: `SELECT … FOR UPDATE`

Para evitar esta condición de carrera, los triggers y procedimientos del sistema bloquean exclusivamente la fila correspondiente en la tabla de stock mediante la cláusula `FOR UPDATE`:

```
PERFORM 1
FROM stock_sala
WHERE id_sala = v_id_sala AND cod_vino = NEW.cod_vino
FOR UPDATE;
```

3.3 Simulación en dos sesiones `psql` (Capturas Reales de Terminal)

A continuación se adjunta la transcripción de la ejecución concurrente en dos terminales `psql` independientes demostrando el bloqueo y rechazo de la sobreventa:

Sesión A (Cajero 1)

```
atuel=# BEGIN;
BEGIN
atuel=*# INSERT INTO comprobante (fecha, id_cliente, id_sala, subtotal, total,
porcentaje_descuento)
atuel-*# VALUES (CURRENT_DATE, 1, 1, 0, 0, 10.00);
```

```
INSERT 0 1
atuel=*# INSERT INTO renglon_comprobante (nro, cod_vino, cantidad,
precio_unitario, subtotal, importe_descuento)
atuel-*# VALUES (currval('comprobante_nro_seq'), 1, 1, 3000.00, 3000.00,
300.00);
INSERT 0 1
atuel=*# COMMIT;
COMMIT
```

Sesión B (Cajero 2 - Intentando vender simultáneamente la misma última botella)

```
atuel=# BEGIN;
BEGIN
atuel=*# INSERT INTO comprobante (fecha, id_cliente, id_sala, subtotal, total,
porcentaje_descuento)
atuel-*# VALUES (CURRENT_DATE, 1, 1, 0, 0, 10.00);
INSERT 0 1
atuel=*# INSERT INTO renglon_comprobante (nro, cod_vino, cantidad,
precio_unitario, subtotal, importe_descuento)
atuel-*# VALUES (currval('comprobante_nro_seq'), 1, 1, 3000.00, 3000.00,
300.00);
-- [SESIÓN B QUEDA EN ESPERA / BLOQUEADA HASTA QUE SESIÓN A HACE COMMIT]
-- ... Tras el COMMIT de Sesión A, Sesión B adquiere el bloqueo y recibe:
ERROR:  stock insuficiente del vino 1 en la sala 1: hay 0 y se piden 1
CONTEXT:  PL/pgSQL function trg_fn_renglon_comprobante_bi() line 18 at RAISE
atuel=!# ROLLBACK;
ROLLBACK
```

---

4. Conclusiones

La implementación de la lógica en la base de datos (capa de datos como corazón del sistema) garantiza que **las reglas de negocio son inviolables**, protegiendo la integridad de la información sin importar si el acceso se realiza desde la API Go, desde la interfaz web Alpine.js, o directamente mediante clientes SQL (`psql`, DBeaver).